

Chapter 7: Temporal-Difference Methods

Adapted from Shiyu Zhao's textbook

Outline

- 1 Introduction to TD Methods
- 2 TD Learning of State Values
- 3 Sarsa: TD Learning of Action Values
- 4 n-step Sarsa
- 5 Q-Learning
- 6 A Unified Viewpoint
- 7 Summary

Where TD Methods Fit in RL

- TD learning is **model-free** (like Monte Carlo)
- TD learning is **incremental** (unlike Monte Carlo)
- TD algorithms are **stochastic approximation** algorithms for solving Bellman equations

Algorithms Covered:

- ① **TD Learning** (Section 7.1)
Estimate state values of a given policy
- ② **Sarsa** (Section 7.2)
Estimate action values of a given policy
- ③ **n-step Sarsa** (Section 7.3)
Generalization of Sarsa
- ④ **Q-learning** (Section 7.4)
Directly solve Bellman optimality equation

Key Relationships:

- Sarsa $\leftarrow$ TD Learning
(state $\rightarrow$ action values)
- n-step Sarsa generalizes both Sarsa and MC
- Q-learning is **off-policy**
(others are on-policy)

TD Learning: Problem Setup

Goal

Given a policy π , estimate $v_\pi(s)$ for all $s \in \mathcal{S}$

Available: Experience samples generated following π :

$$(s_0, r_1, s_1, \dots, s_t, r_{t+1}, s_{t+1}, \dots)$$

Key Insight

TD learning can estimate state values using these samples **incrementally**, without waiting for complete episodes.

TD Learning Algorithm

TD Learning Update Rule

$$v_{t+1}(s_t) = v_t(s_t) - \alpha_t(s_t) \left[v_t(s_t) - (r_{t+1} + \gamma v_t(s_{t+1})) \right] \quad (1)$$

$$v_{t+1}(s) = v_t(s), \quad \text{for all } s \neq s_t \quad (2)$$

Notation:

- $v_t(s_t)$: estimate of $v_\pi(s_t)$ at time t
- $\alpha_t(s_t)$: learning rate for s_t at time t
- γ : discount factor

Important: At time t , only the visited state s_t is updated. Unvisited states remain unchanged.

TD Algorithm Structure

The TD algorithm can be rewritten as:

$$\underbrace{v_{t+1}(s_t)}_{\text{new estimate}} = \underbrace{v_t(s_t)}_{\text{current estimate}} - \alpha_t(s_t) \underbrace{\left(v_t(s_t) - \underbrace{(r_{t+1} + \gamma v_t(s_{t+1}))}_{\text{TD target } \bar{v}_t} \right)}_{\text{TD error } \delta_t}$$

TD Target

$$\bar{v}_t \triangleq r_{t+1} + \gamma v_t(s_{t+1})$$

The value we're driving $v(s_t)$ toward

TD Error

$$\delta_t \triangleq v_t(s_t) - \bar{v}_t$$

Discrepancy between time steps t and $t + 1$

Why is $\bar{v}_t$ Called the TD Target?

Proof that the algorithm drives $v(s_t)$ toward $\bar{v}_t$:

Subtracting $\bar{v}_t$ from both sides of the update:

$$\begin{aligned}v_{t+1}(s_t) - \bar{v}_t &= v_t(s_t) - \bar{v}_t - \alpha_t(s_t)(v_t(s_t) - \bar{v}_t) \\ &= [1 - \alpha_t(s_t)](v_t(s_t) - \bar{v}_t)\end{aligned}$$

Taking absolute values:

$$|v_{t+1}(s_t) - \bar{v}_t| = |1 - \alpha_t(s_t)| |v_t(s_t) - \bar{v}_t|$$

Since $\alpha_t(s_t)$ is a small positive number, we have $0 < 1 - \alpha_t(s_t) < 1$.

Key Result

$$|v_{t+1}(s_t) - \bar{v}_t| < |v_t(s_t) - \bar{v}_t|$$

The new value is **closer** to $\bar{v}_t$ than the old value!

Interpretation of the TD Error

TD Error

$$\delta_t = v_t(s_t) - (r_{t+1} + \gamma v_t(s_{t+1}))$$

Two Interpretations:

- ① **Temporal Difference:** δ_t reflects discrepancy between time steps t and $t + 1$
- ② **Estimation Error:** When $v_t = v_\pi$, the expected TD error is zero:

$$\begin{aligned}\mathbb{E}[\delta_t | S_t = s_t] &= \mathbb{E}[v_\pi(S_t) - (R_{t+1} + \gamma v_\pi(S_{t+1})) | S_t = s_t] \\ &= v_\pi(s_t) - \mathbb{E}[R_{t+1} + \gamma v_\pi(S_{t+1}) | S_t = s_t] \\ &= 0 \quad (\text{by the Bellman equation})\end{aligned}$$

Innovation: TD error represents **new information** from sample (s_t, r_{t+1}, s_{t+1})

Derivation: TD as Stochastic Approximation

Step 1: Recall the definition of state value:

$$v_{\pi}(s) = \mathbb{E}[R_{t+1} + \gamma G_{t+1} | S_t = s]$$

Step 2: Rewrite using $\mathbb{E}[G_{t+1} | S_t = s] = \mathbb{E}[v_{\pi}(S_{t+1}) | S_t = s]$:

$$v_{\pi}(s) = \mathbb{E}[R_{t+1} + \gamma v_{\pi}(S_{t+1}) | S_t = s]$$

This is the **Bellman expectation equation**.

Step 3: Define the function to solve:

$$g(v_{\pi}(s_t)) \triangleq v_{\pi}(s_t) - \mathbb{E}[R_{t+1} + \gamma v_{\pi}(S_{t+1}) | S_t = s_t] = 0$$

Step 4: Apply Robbins-Monro algorithm to solve $g(v_{\pi}(s_t)) = 0$

Derivation: TD as Stochastic Approximation (cont.)

Step 5: The noisy observation of $g(v_\pi(s_t))$ is:

$$\begin{aligned}\tilde{g}(v_\pi(s_t)) &= v_\pi(s_t) - (r_{t+1} + \gamma v_\pi(s_{t+1})) \\ &= \underbrace{v_\pi(s_t) - \mathbb{E}[R_{t+1} + \gamma v_\pi(S_{t+1}) | S_t = s_t]}_{g(v_\pi(s_t))} \\ &\quad + \underbrace{\mathbb{E}[R_{t+1} + \gamma v_\pi(S_{t+1}) | S_t = s_t] - (r_{t+1} + \gamma v_\pi(s_{t+1}))}_{\text{noise } \eta}\end{aligned}$$

Step 6: Robbins-Monro algorithm:

$$v_{t+1}(s_t) = v_t(s_t) - \alpha_t(s_t) \tilde{g}(v_t(s_t)) = v_t(s_t) - \alpha_t(s_t) [v_t(s_t) - (r_{t+1} + \gamma v_t(s_{t+1}))]$$

Result

This is exactly the TD learning algorithm! TD learning is a stochastic approximation algorithm for solving the Bellman equation.

Convergence of TD Learning

Theorem (Convergence of TD Learning)

Given a policy π , by the TD algorithm, $v_t(s)$ converges almost surely to $v_\pi(s)$ as $t \rightarrow \infty$ for all $s \in \mathcal{S}$ if:

$$\sum_t \alpha_t(s) = \infty \quad \text{and} \quad \sum_t \alpha_t^2(s) < \infty \quad \text{for all } s \in \mathcal{S}$$

Remarks on α_t :

- $\alpha_t(s) > 0$ if s is being visited at time t ; otherwise $\alpha_t(s) = 0$
- $\sum_t \alpha_t(s) = \infty$ requires state s to be visited infinitely often
- This requires **exploring starts** or an **exploratory policy**
- In practice, α_t is often a small **constant** (then convergence is in expectation)

TD Learning vs Monte Carlo Learning

Property	TD Learning	MC Learning
Update Style	Incremental : updates immediately after each sample	Non-incremental: must wait for complete episode
Task Type	Episodic and continuing tasks	Episodic tasks only
Bootstrapping	Yes: relies on previous estimates; requires initial guess	No: directly estimates from returns
Variance	Lower : fewer random variables involved	Higher: many random variables in return

Variance Example:

- TD (Sarsa): only needs samples of $R_{t+1}, S_{t+1}, A_{t+1}$
- MC: needs $R_{t+1} + \gamma R_{t+2} + \gamma^2 R_{t+3} + \dots$
- With $|\mathcal{A}|$ actions and episode length L : $|\mathcal{A}|^L$ possible episodes!

Limitation of TD Learning

The TD algorithm in Section 7.1 can only estimate **state values**.

To find optimal policies, we need:

- 1 Calculate **action values**
- 2 Conduct **policy improvement**

Sarsa Solution

Sarsa directly estimates **action values** $q_{\pi}(s, a)$ using TD learning.

Combined with policy improvement, Sarsa can learn optimal policies!

Sarsa Algorithm

Goal: Given policy π , estimate action values $q_\pi(s, a)$

Experience samples: $(s_0, a_0, r_1, s_1, a_1, \dots, s_t, a_t, r_{t+1}, s_{t+1}, a_{t+1}, \dots)$

Sarsa Update Rule

$$q_{t+1}(s_t, a_t) = q_t(s_t, a_t) - \alpha_t(s_t, a_t) \left[q_t(s_t, a_t) - (r_{t+1} + \gamma q_t(s_{t+1}, a_{t+1})) \right]$$

$$q_{t+1}(s, a) = q_t(s, a), \quad \text{for all } (s, a) \neq (s_t, a_t)$$

Why “Sarsa”?

- Each iteration requires: $(s_t, a_t, r_{t+1}, s_{t+1}, a_{t+1})$
- **State-Action-Reward-State-Action = SARSA**

Sarsa: Mathematical Foundation

What equation does Sarsa solve?

Bellman Equation for Action Values

$$q_{\pi}(s, a) = \mathbb{E}[R + \gamma q_{\pi}(S', A') | s, a], \quad \text{for all } (s, a)$$

Proof: The Bellman equation in terms of action values is:

$$q_{\pi}(s, a) = \sum_r r \cdot p(r | s, a) + \gamma \sum_{s'} p(s' | s, a) \sum_{a'} q_{\pi}(s', a') \pi(a' | s')$$

Since $p(s', a' | s, a) = p(s' | s, a) \cdot \pi(a' | s')$, we can write:

$$q_{\pi}(s, a) = \sum_r r \cdot p(r | s, a) + \gamma \sum_{s'} \sum_{a'} q_{\pi}(s', a') p(s', a' | s, a)$$

By the definition of expectation, this is $\mathbb{E}[R + \gamma q_{\pi}(S', A') | s, a]$.

Convergence of Sarsa

Theorem (Convergence of Sarsa)

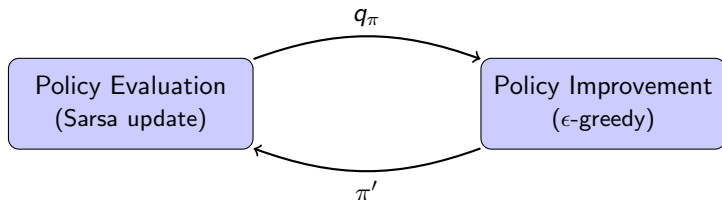
Given a policy π , by the Sarsa algorithm, $q_t(s, a)$ converges almost surely to $q_\pi(s, a)$ as $t \rightarrow \infty$ for all (s, a) if:

$$\sum_t \alpha_t(s, a) = \infty \quad \text{and} \quad \sum_t \alpha_t^2(s, a) < \infty \quad \text{for all } (s, a)$$

Key Requirements:

- $\sum_t \alpha_t(s, a) = \infty$ requires every state-action pair to be visited infinitely often
- At time t : $\alpha_t(s, a) > 0$ if $(s, a) = (s_t, a_t)$, otherwise $\alpha_t(s, a) = 0$
- Exploration is essential!

Sarsa for Optimal Policy Learning



Key Idea: Generalized Policy Iteration

- Don't wait until policy is fully evaluated
- Update policy after each q-value update
- Use ϵ -greedy policy for exploration

Algorithm: Optimal Policy Learning by Sarsa

Sarsa Algorithm

Initialize: $\alpha > 0$, $\epsilon \in (0, 1)$, $q_0(s, a)$ for all (s, a) , ϵ -greedy policy π_0 from q_0

For each episode:

- ① Generate a_0 at s_0 following $\pi_0(s_0)$
- ② **While** s_t is not terminal:
 - ① Collect experience $(r_{t+1}, s_{t+1}, a_{t+1})$:
 - Generate r_{t+1}, s_{t+1} by interacting with environment
 - Generate a_{t+1} following $\pi_t(s_{t+1})$
 - ② Update q-value:

$$q_{t+1}(s_t, a_t) = q_t(s_t, a_t) - \alpha [q_t(s_t, a_t) - (r_{t+1} + \gamma q_t(s_{t+1}, a_{t+1}))]$$

- ③ Update policy to ϵ -greedy based on q_{t+1}
- ④ $s_t \leftarrow s_{t+1}$, $a_t \leftarrow a_{t+1}$

Sarsa Example: Grid World

Setup:

- 5×5 grid world
- Start: top-left state
- Goal: reach target (blue cell)
- Rewards:
 - $r_{\text{target}} = 0$
 - $r_{\text{forbidden}} = r_{\text{boundary}} = -10$
 - $r_{\text{other}} = -1$
- Parameters: $\alpha = 0.1$, $\epsilon = 0.1$

Results:

- Policy successfully leads to target
- Total reward increases over episodes
- Episode length decreases
- Occasional spikes due to ϵ -greedy exploration

Note: Policies for unvisited states may not be optimal (insufficient exploration)

n-step Sarsa: Motivation

Recall the definition of action value:

$$q_{\pi}(s, a) = \mathbb{E}[G_t | S_t = s, A_t = a]$$

where $G_t = R_{t+1} + \gamma R_{t+2} + \gamma^2 R_{t+3} + \dots$

Key Insight: G_t can be decomposed in different ways!

$$\text{Sarsa} \leftarrow G_t^{(1)} = R_{t+1} + \gamma q_{\pi}(S_{t+1}, A_{t+1})$$

$$G_t^{(2)} = R_{t+1} + \gamma R_{t+2} + \gamma^2 q_{\pi}(S_{t+2}, A_{t+2})$$

$$\vdots$$

$$\text{n-step Sarsa} \leftarrow G_t^{(n)} = R_{t+1} + \gamma R_{t+2} + \dots + \gamma^n q_{\pi}(S_{t+n}, A_{t+n})$$

$$\vdots$$

$$\text{MC} \leftarrow G_t^{(\infty)} = R_{t+1} + \gamma R_{t+2} + \gamma^2 R_{t+3} + \dots$$

Note: $G_t = G_t^{(1)} = G_t^{(2)} = \dots = G_t^{(n)} = G_t^{(\infty)}$ (different decomposition structures)

n-step Sarsa: Special Cases

Case 1: $n = 1$ (Sarsa)

$$q_{\pi}(s, a) = \mathbb{E}[R_{t+1} + \gamma q_{\pi}(S_{t+1}, A_{t+1}) | s, a]$$

Algorithm: $q_{t+1}(s_t, a_t) = q_t(s_t, a_t) - \alpha [q_t(s_t, a_t) - (r_{t+1} + \gamma q_t(s_{t+1}, a_{t+1}))]$

Case 2: $n = \infty$ (Monte Carlo)

$$q_{\pi}(s, a) = \mathbb{E}[R_{t+1} + \gamma R_{t+2} + \gamma^2 R_{t+3} + \dots | s, a]$$

Algorithm: $q_{t+1}(s_t, a_t) = g_t = r_{t+1} + \gamma r_{t+2} + \gamma^2 r_{t+3} + \dots$

Unification

Sarsa and MC learning are two extreme cases of n-step Sarsa!

n-step Sarsa Algorithm

General case: arbitrary n

$$q_{\pi}(s, a) = \mathbb{E}[R_{t+1} + \gamma R_{t+2} + \cdots + \gamma^n q_{\pi}(S_{t+n}, A_{t+n}) | s, a]$$

n-step Sarsa Update

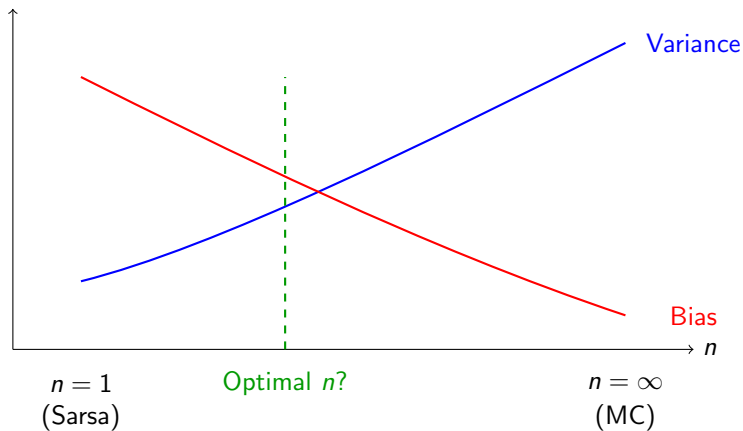
$$q_{t+1}(s_t, a_t) = q_t(s_t, a_t) - \alpha_t(s_t, a_t) \left[q_t(s_t, a_t) - (r_{t+1} + \gamma r_{t+2} + \cdots + \gamma^n q_t(s_{t+n}, a_{t+n})) \right]$$

Implementation Note:

- Need experience samples: $(s_t, a_t, r_{t+1}, s_{t+1}, a_{t+1}, \dots, r_{t+n}, s_{t+n}, a_{t+n})$
- Must wait until time $t + n$ to update $q(s_t, a_t)$
- Rewritten update at time $t + n$:

$$q_{t+n}(s_t, a_t) = q_{t+n-1}(s_t, a_t) - \alpha [q_{t+n-1}(s_t, a_t) - (r_{t+1} + \gamma r_{t+2} + \cdots + \gamma^n q_{t+n-1}(s_{t+n}, a_{t+n}))]$$

n-step Sarsa: Bias-Variance Tradeoff



Small n (close to Sarsa):

- Large bias
- Low variance

Large n (close to MC):

- Small bias
- High variance

Q-Learning: Introduction

Key Difference from Sarsa

- **Sarsa**: Estimates action values of a given policy; needs policy improvement
- **Q-learning**: Directly estimates optimal action values and finds optimal policies

Historical Note

Q-learning is one of the most classic reinforcement learning algorithms, introduced by Watkins (1989).

Key Innovation: Q-learning solves the **Bellman optimality equation** rather than the Bellman equation of a given policy.

Q-Learning Algorithm

Q-Learning Update Rule

$$q_{t+1}(s_t, a_t) = q_t(s_t, a_t) - \alpha_t(s_t, a_t) \left[q_t(s_t, a_t) - (r_{t+1} + \gamma \max_{a \in \mathcal{A}(s_{t+1})} q_t(s_{t+1}, a)) \right]$$
$$q_{t+1}(s, a) = q_t(s, a), \quad \text{for all } (s, a) \neq (s_t, a_t)$$

Comparison with Sarsa:

	Sarsa	Q-learning
TD Target	$r_{t+1} + \gamma q_t(s_{t+1}, a_{t+1})$	$r_{t+1} + \gamma \max_a q_t(s_{t+1}, a)$
Samples Needed	$(s_t, a_t, r_{t+1}, s_{t+1}, a_{t+1})$	$(s_t, a_t, r_{t+1}, s_{t+1})$

Q-learning does **not** need a_{t+1} — only (r_{t+1}, s_{t+1}) given (s_t, a_t) !

Q-Learning: Mathematical Foundation

What equation does Q-learning solve?

Bellman Optimality Equation (Action Values)

$$q(s, a) = \mathbb{E} \left[R_{t+1} + \gamma \max_a q(S_{t+1}, a) \middle| S_t = s, A_t = a \right]$$

Proof: By the definition of expectation:

$$q(s, a) = \sum_r p(r|s, a) \cdot r + \gamma \sum_{s'} p(s'|s, a) \max_{a \in \mathcal{A}(s')} q(s', a)$$

Taking max over both sides and letting $v(s) \triangleq \max_a q(s, a)$:

$$v(s) = \max_a \left[\sum_r p(r|s, a) \cdot r + \gamma \sum_{s'} p(s'|s, a) v(s') \right]$$

This is the **Bellman optimality equation** in terms of state values!

On-Policy vs Off-Policy Learning

Definitions

- **Behavior policy** π_b : Policy used to generate experience samples
- **Target policy** π_T : Policy being learned/improved

On-Policy

$$\pi_b = \pi_T$$

The policy generating samples is the same as the policy being learned.

Examples: Sarsa, MC Learning

Off-Policy

$$\pi_b \neq \pi_T$$

Can learn from samples generated by a different policy.

Example: Q-learning

Advantage of Off-Policy: Can learn from experience generated by other policies (e.g., human operators, exploratory policies)

Why is Sarsa On-Policy?

Sample generation in Sarsa:

$$s_t \xrightarrow{\pi_b} a_t \xrightarrow{\text{model}} r_{t+1}, s_{t+1} \xrightarrow{\pi_b} a_{t+1}$$

- Sarsa evaluates policy π_T by solving its Bellman equation
- The evaluation requires samples $(r_{t+1}, s_{t+1}, a_{t+1})$
- a_{t+1} is generated by π_b
- The policy being evaluated (π_T) must equal the policy generating a_{t+1} (π_b)
- Therefore: $\pi_T = \pi_b$ (on-policy)

In other words: The policy that Sarsa evaluates **is** the policy used to generate samples.

Why is Q-Learning Off-Policy?

Sample generation in Q-learning:

$$s_t \xrightarrow{\pi_b} a_t \xrightarrow{\text{model}} r_{t+1}, s_{t+1}$$

- Q-learning solves the **Bellman optimality equation**
- Samples needed: $(s_t, a_t, r_{t+1}, s_{t+1})$
- (r_{t+1}, s_{t+1}) are generated by the environment/model, not by π_b
- The estimation of optimal action values does not depend on π_b
- We can use any π_b to generate a_t at s_t

Target policy: π_T is the greedy policy based on estimated optimal values

Behavior policy: π_b can be any exploratory policy

Since $\pi_b \neq \pi_T$ is allowed, Q-learning is **off-policy**.

Online vs Offline Learning

Don't Confuse with On-Policy/Off-Policy!

- **Online learning:** Agent updates values/policies while interacting with environment
- **Offline learning:** Agent updates using pre-collected data, without interaction

	Online	Offline
On-policy	✓	× (needs same policy)
Off-policy	✓	✓

Key insight:

- On-policy algorithms must be online (can't use pre-collected data from other policies)
- Off-policy algorithms can be either online or offline

Q-Learning: On-Policy Implementation

Algorithm (On-Policy Version)

Initialize: $\alpha > 0$, $\epsilon \in (0, 1)$, $q_0(s, a)$ for all (s, a) , ϵ -greedy policy π_0 from q_0

For each episode, while s_t is not terminal:

① Collect experience (a_t, r_{t+1}, s_{t+1}) :

- Generate a_t following $\pi_t(s_t)$
- Generate r_{t+1}, s_{t+1} by interacting with environment

② Update q-value:

$$q_{t+1}(s_t, a_t) = q_t(s_t, a_t) - \alpha [q_t(s_t, a_t) - (r_{t+1} + \gamma \max_a q_t(s_{t+1}, a))]$$

③ Update policy to ϵ -greedy based on q_{t+1}

Note: Behavior policy = Target policy = ϵ -greedy (but Q-learning is still fundamentally off-policy!)

Q-Learning: Off-Policy Implementation

Algorithm (Off-Policy Version)

Initialize: $q_0(s, a)$ for all (s, a) , behavior policy $\pi_b(a|s)$, $\alpha > 0$

For each episode $\{s_0, a_0, r_1, s_1, a_1, r_2, \dots\}$ **generated by** π_b :

For each step $t = 0, 1, 2, \dots$ **of the episode:**

- 1 Update q-value:

$$q_{t+1}(s_t, a_t) = q_t(s_t, a_t) - \alpha [q_t(s_t, a_t) - (r_{t+1} + \gamma \max_a q_t(s_{t+1}, a))]$$

- 2 Update target policy (greedy):

$$\pi_{T,t+1}(a|s_t) = \begin{cases} 1 & \text{if } a = \arg \max_a q_{t+1}(s_t, a) \\ 0 & \text{otherwise} \end{cases}$$

Key differences:

- Behavior policy π_b can be any exploratory policy
- Target policy π_T is greedy (not ϵ -greedy) since it doesn't generate samples

Q-Learning Example: Grid World

On-Policy Q-Learning:

- Same setup as Sarsa example
- Start: top-left, Goal: target cell
- $r_{\text{target}} = 0$, $r_{\text{forbidden}} = r_{\text{boundary}} = -10$, $r_{\text{other}} = -1$
- $\alpha = 0.1$, $\epsilon = 0.1$

Results:

- Successfully finds optimal path
- Episode length decreases
- Total reward increases

Off-Policy Q-Learning:

- Goal: optimal policy for all states
- Rewards: $r_{\text{boundary}} = r_{\text{forbidden}} = -1$, $r_{\text{target}} = 1$
- Discount: $\gamma = 0.9$, $\alpha = 0.1$
- Single episode with 100,000 steps

Behavior policy: Uniform distribution (highly exploratory)

Effect of Initial Values on Q-Learning

Q-learning bootstraps, so performance depends on initial guess!

Initial $q_0(s, a)$	Convergence Speed
$q_0(s, a) = 0$	$\sim 10,000$ steps
$q_0(s, a) = 10$	$\sim 10,000$ steps (similar)
$q_0(s, a) = 100$	Much slower (high initial error)

Key observation: Q-learning converges even with poor initial values, but convergence is faster when initial values are close to true values.

Effect of Behavior Policy on Q-Learning

Exploration is critical for off-policy Q-learning!

Behavior Policy	ϵ value	Learning Speed
Uniform	$\epsilon = 1$	Fast (good exploration)
ϵ -greedy	$\epsilon = 0.5$	Slower
ϵ -greedy	$\epsilon = 0.1$	Much slower (poor exploration)

Why?

- Less exploratory $\rightarrow$ fewer state-action pairs visited
- Insufficient samples $\rightarrow$ slow/incomplete learning
- Off-policy advantage only works if behavior policy provides good coverage!

Unified Expression for TD Algorithms

All TD algorithms (for action value estimation) can be expressed as:

Unified TD Update

$$q_{t+1}(s_t, a_t) = q_t(s_t, a_t) - \alpha_t(s_t, a_t) [q_t(s_t, a_t) - \bar{q}_t]$$

where $\bar{q}_t$ is the **TD target**.

Algorithm	TD Target $\bar{q}_t$
Sarsa	$r_{t+1} + \gamma q_t(s_{t+1}, a_{t+1})$
n-step Sarsa	$r_{t+1} + \gamma r_{t+2} + \dots + \gamma^n q_t(s_{t+n}, a_{t+n})$
Q-learning	$r_{t+1} + \gamma \max_a q_t(s_{t+1}, a)$
Monte Carlo	$r_{t+1} + \gamma r_{t+2} + \gamma^2 r_{t+3} + \dots$

MC as special case: Set $\alpha_t = 1$, then $q_{t+1}(s_t, a_t) = \bar{q}_t = g_t$

Equations Solved by TD Algorithms

Unified view: Each algorithm is stochastic approximation for $q(s, a) = \mathbb{E}[\bar{q}_t | s, a]$

Algorithm	Equation Solved	Type
Sarsa	$q_\pi(s, a) = \mathbb{E}[R_{t+1} + \gamma q_\pi(S_{t+1}, A_{t+1}) s, a]$	BE
n-step Sarsa	$q_\pi(s, a) = \mathbb{E}[R_{t+1} + \dots + \gamma^n q_\pi(S_{t+n}, A_{t+n}) s, a]$	BE
Q-learning	$q(s, a) = \mathbb{E}[R_{t+1} + \gamma \max_a q(S_{t+1}, a) s, a]$	BOE
MC	$q_\pi(s, a) = \mathbb{E}[R_{t+1} + \gamma R_{t+2} + \gamma^2 R_{t+3} + \dots s, a]$	BE

Legend:

- BE = Bellman Equation (for a given policy π)
- BOE = Bellman Optimality Equation (for optimal values)

Key insight: Q-learning is the only one solving BOE directly!

Summary: Algorithm Comparison

	Sarsa	n-step Sarsa	Q-learning	MC
Estimates	q_π	q_π	q^*	q_π
Incremental	Yes	Yes	Yes	No
Bootstraps	Yes	Yes	Yes	No
Policy type	On-policy	On-policy	Off-policy	On-policy
Task type	Episodic/Cont.	Episodic/Cont.	Episodic/Cont.	Episodic
Variance	Low	Medium	Low	High
Bias	Yes	Medium	Yes	No

To learn optimal policies:

- Sarsa, n-step Sarsa, MC: Need policy improvement step
- Q-learning: Directly obtains optimal policy (greedy w.r.t. q^*)

Chapter Summary

Key Algorithms Introduced:

- 1 **TD Learning:** Basic algorithm for state value estimation
- 2 **Sarsa:** TD learning for action values (on-policy)
- 3 **n-step Sarsa:** Generalizes Sarsa and MC (adjustable bias-variance)
- 4 **Q-learning:** Directly learns optimal values (off-policy)

Core Insights:

- All TD algorithms are stochastic approximation algorithms
- Sarsa, n-step Sarsa: Solve Bellman equation $\rightarrow$ need policy improvement
- Q-learning: Solves Bellman optimality equation $\rightarrow$ direct optimal policy
- Q-learning is off-policy because it doesn't depend on behavior policy

Key Takeaways

- ① **TD = Temporal Difference**: Error reflects discrepancy between time steps
- ② **Learning = Estimation**: Estimate values from samples, then derive policies
- ③ **Sarsa + Policy Improvement = Optimal Policy**: Use generalized policy iteration
- ④ **ϵ -greedy for exploration**: On-policy methods need exploratory policies
- ⑤ **Constant α in practice**: Because policy is nonstationary during learning
- ⑥ **Q-learning is off-policy**: Solves BOE, not BE of a given policy
- ⑦ **Off-policy $\neq$ Offline**: Off-policy can be online or offline; on-policy must be online

Extensions and Further Reading

Variants and Extensions:

- **TD(λ)**: Unified framework with eligibility traces
- **Expected Sarsa**: Lower variance than Sarsa
- **Double Q-learning**: Reduces overestimation bias
- **Importance Sampling**: Convert on-policy to off-policy

Next Chapters:

- Chapter 8: Value Function Approximation (beyond tabular)
- Chapter 9: Policy Gradient Methods
- Chapter 10: Actor-Critic Methods

Appendix: Proof Sketch of TD Convergence

Based on Theorem 6.3 (Stochastic Approximation)

Define estimation error: $\Delta_t(s) \triangleq v_t(s) - v_\pi(s)$

The TD update becomes:

$$\Delta_{t+1}(s) = (1 - \alpha_t(s))\Delta_t(s) + \alpha_t(s)\eta_t(s)$$

where $\eta_t(s) = r_{t+1} + \gamma v_t(s_{t+1}) - v_\pi(s)$ for $s = s_t$, and $\eta_t(s) = 0$ otherwise.

Three conditions to verify:

- ① $\sum_t \alpha_t(s) = \infty$, $\sum_t \alpha_t^2(s) < \infty$ (assumed)
- ② $\|\mathbb{E}[\eta_t(s)|H_t]\|_\infty \leq \gamma\|\Delta_t(s)\|_\infty$ (contraction property)
- ③ Bounded variance (follows from bounded rewards)

Condition 2 uses: $|\mathbb{E}[\eta_t(s)]| = \gamma|\sum_{s'} p(s'|s_t)[v_t(s') - v_\pi(s')]| \leq \gamma\|\Delta_t\|_\infty$

Appendix: Complete Sarsa Derivation

Starting point: Bellman equation for action values

$$q_{\pi}(s, a) = \sum_r p(r|s, a) \cdot r + \gamma \sum_{s'} p(s'|s, a) \sum_{a'} \pi(a'|s') q_{\pi}(s', a')$$

Key observation:

$$p(s', a'|s, a) = p(s'|s, a) \cdot p(a'|s', s, a) = p(s'|s, a) \cdot \pi(a'|s')$$

(by conditional independence: A' depends only on S' , not on (S, A))

Rewriting:

$$q_{\pi}(s, a) = \sum_r p(r|s, a) \cdot r + \gamma \sum_{s'} \sum_{a'} p(s', a'|s, a) q_{\pi}(s', a')$$

By definition of expectation:

$$q_{\pi}(s, a) = \mathbb{E}[R + \gamma q_{\pi}(S', A') | S = s, A = a]$$